

Electron 通信模板（v1 + v2，JS 版本）

一、总体结构

main.js → 主进程（Electron + Node）

preload.js → 桥接层（Electron + Node → 浏览器）

React → 渲染进程（浏览器环境）

二、v1：基础通信模型（请求-响应）

main.js

```
const { app, BrowserWindow, ipcMain, dialog } = require('electron');
const path = require('path');
const fs = require('fs');

function createWindow() {
  const win = new BrowserWindow({
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
      contextIsolation: true,
      nodeIntegration: false,
    },
  });
}

win.loadURL(process.env.MAIN_WINDOW_VITE_DEV_SERVER_URL);

app.whenReady().then(createWindow);

ipcMain.handle('app:get-version', () => app.getVersion());

ipcMain.handle('file:read-text', async () => {
  const result = await dialog.showOpenDialog({ properties: ['openFile'] });
  if (result.canceled) return null;
  return fs.readFileSync(result.filePaths[0], 'utf-8');
});
```

preload.js

```
const { contextBridge, ipcRenderer } = require('electron');
```

```
contextBridge.exposeInMainWorld('electronAPI', {
  getAppVersion: () => ipcRenderer.invoke('app:get-version'),
  readTextFile: () => ipcRenderer.invoke('file:read-text'),
});
```

React 使用

```
const v = await window.electronAPI.getAppVersion();
```

三、v2：完整通信模型

1. renderer → main（无返回值）

```
// preload
sendLog: (msg) => ipcRenderer.send('log:message', msg)

// main
ipcMain.on('log:message', (e, msg) => console.log(msg));
```

2. main → renderer（主动推送）

```
// main
win.webContents.send('timer:tick', count);

// preload
onTimerTick: (cb) => ipcRenderer.on('timer:tick', (e, c) => cb(c));
```

3. 多窗口通信

```
// main
secondWindow.webContents.send('message:from-main', msg);

// preload
onMessage: (cb) => ipcRenderer.on('message:from-main', (e, m) => cb(m));
```

四、通信链路总结

v1（请求）

React → preload → main → Node/Electron → 返回结果

v2（事件）

React → preload → main（send） main → renderer（主动 send）

五、核心原则

- React 不直接使用 Node.js
 - 所有系统能力必须走 preload
 - preload 只暴露“功能”，不暴露底层 API
 - main 才是真正执行系统操作的地方
-

六、一句话总结

v1: React 向 main 请求数据
v2: React 和 main 双向通信（事件 + 推送 + 多窗口）